Compte rendu – TP 2 : BFS et 8-Puzzle

Partie 1 – Compréhension du problème

1.

- Etat initial : La configuration de départ du puzzle {1,2,3,8,0,4,7,6,5}.
- Etat but :La configuration finale à atteindre {1,2,3,4,5,6,7,8,0}.
- Les Opérateurs :
  - Déplacer la case vide (0) à droite
  - Déplacer la case vide (0) à gauche
  - Déplacer la case vide (0) en haut
  - Déplacer la case vide (0) en bas
- Le test-but:
  Vérifier si la configuration actuelle est égale à l'état but `(Arrays.equals)` . Dans le code, il est implémenté par la méthode `isGoal(Node node)` de la classe `BreadthFirstSearch`, qui compare le tableau `puzzle` du nœud courant avec le tableau `goal` préenregistré.

2. Le 8-Puzzle est un problème de recherche dans un espace d'états car :

- Un ensemble fini d'opérateurs qui permettent de naviguer d'un état à un autre
- chaque configuration représente un état(Un état initial clairement défini et un état but).
- l'ensemble des configurations possibles forme un espace d'états,
- Le but est donc de trouver une séquence d'opérateurs (un chemin) qui mène de l'état initial à l'état but.

Partie 2 – Compréhension du code

3. `La classe Node` représente un état du puzzle dans l'arbre de recherche.

- Rôle : Elle encapsule la configuration courante du taquin (dans l'attribut int[] puzzle) et sert d'élément de base pour la construction de l'arbre de recherche.

`L'attribut parent` permet de reconstruire le chemin solution en remontant depuis l'état but jusqu'à l'état initial.

4.`La structure visited` est indispensable pour :

- Éviter les boucles infinies : Sans visited, l'algorithme pourrait, par exemple, déplacer la case vide à droite, puis à gauche, et se retrouver dans l'état précédent, puis recommencer indéfiniment, créant ainsi des cycles dans l'arbre de recherche.
- Éviter les redondances : Un même état peut être atteint par différents chemins de longueurs différentes. Sans visited, cet état serait exploré plusieurs fois, ce qui est inutile et augmente considérablement le temps de calcul et l'utilisation mémoire. En marquant les états comme visités dès leur première rencontre, on garantit que chaque état n'est exploré qu'une seule fois.

Partie 3 – Analyse algorithmique

5. BFS garantit une solution optimale car il explore les états niveau par niveau. La première fois que l'état but est trouvé, il correspond au nombre minimal de mouvements.
6. Le nombre d'itérations peut être très élevé car BFS explore tous les états d'un niveau avant de passer au suivant, même si la solution finale est proche.
7. Complexité approximative :
- Temps : O(b^d)
- Mémoire : O(b^d) où b est le facteur de branchement et d la profondeur de la solution.

Partie 4 – Réflexion critique

8.

Critère	BFS (Recherche en Largeur)	DFS (Recherche en Profondeur)
Optimalité	Optimale car il explore par niveau de profondeur croissante	Non optimale. Elle peut trouver une solution très longue avant une solution courte située dans une autre branche.
Mémoire	Très coûteuse (O(b^d)). Doit stocker tout le niveau frontal.	Très économe (O(b*d)).Ne stocke que le chemin courant et les successeurs à explorer.
Risque de boucle	Le risque est éliminé grâce à la structure visited.	Risque très élevé de cycles infinis

9. BFS est adaptée au 8-Puzzle car l'espace d'états reste gérable.
Pour le 15-Puzzle, le nombre d'états est beaucoup plus grand, ce qui rend BFS trop coûteuse en mémoire.
10. Amélioration possible : Utiliser une recherche informée comme A* avec une heuristique (ex : distance de Manhattan) pour réduire le nombre d'états explorés.

Partie 5 – Expérimentation

11. En modifiant l'état initial :
- le nombre d'états explorés varie fortement,
- plus l'état initial est éloigné du but, plus BFS explore d'états,
- le nombre de mouvements augmente selon la distance réelle au but

12. Pour un état non soluble, le programme explore tous les états possibles puis retourne "Aucune solution trouvée".
Amélioration : Vérifier la solvabilité avant la recherche (calcul du nombre d'inversions) pour éviter une recherche inutile.

Question de synthèse

13. Ce TP illustre les notions fondamentales :
- Espace d'états : La modélisation du problème a montré comment un jeu concret peut être abstrait en un ensemble discret de configurations (les états) reliées par des actions (les opérateurs).
- Recherche aveugle : BFS a mis en évidence le principe d'une exploration systématique et non informée de l'espace d'états, où l'ordre d'exploration est dicté uniquement par la structure des données (la file FIFO).
- Optimalité : BFS garantit le plus court chemin.
- Complexité : le nombre d'états peut croître exponentiellement, ce qui montre les limites pratiques des algorithmes non informés.